

## HANDS-ON 8 [Advanced]

### RESTful API Design Best Practices

Topics Covered:

REST Principles✓

HTTP Methods & Status Codes✓

Resource Naming Conventions✓

API Versioning✓

Pagination & Filtering✓

Error Response Standards✓

Good API design is as important as working code. In this hands-on, you will audit and refactor your existing API endpoints to follow REST best practices — consistent naming, proper status codes, versioning, and standardised error responses.

#### NOTE

This hands-on applies to your Django, Flask, or FastAPI implementation. Pick one and refactor it to meet all the criteria below.

#### Task 1: Audit and Fix Resource Naming and HTTP Methods

Goal: Review existing endpoints and correct any REST convention violations.

Steps:

78.

Review all your existing endpoints. Identify and fix any violations of REST resource naming: URLs should use nouns not verbs (/api/courses/ not /api/getCourses/), should be plural, and should use hyphens not underscores for multi-word resources.

79.

Verify HTTP methods are semantically correct: GET (read, no side effects), POST (create), PUT (full replace), PATCH (partial update), DELETE (remove). Add a PATCH /api/courses/{id}/ endpoint for partial updates alongside the existing PUT.

80.

Verify status codes: 200 OK for GET/PUT/PATCH, 201 Created for POST (with a Location header pointing to the new resource), 204 No Content for DELETE, 400 Bad Request for validation errors, 401 Unauthorised for missing auth, 404 Not Found, 422 Unprocessable Entity for schema errors.

81.

Add a Location response header to all POST endpoints:

```
response.headers['Location'] = f'/api/courses/{new_course.id}'.
```

Hint:

- 

A good rule: if you can read the URL and know what resource it refers to without documentation, the naming is correct.

- 

PUT replaces the entire resource (all fields required). PATCH updates only the supplied fields (all fields optional). Use the right one.

Expected Outcome: All endpoints use plural nouns. PATCH is implemented alongside PUT. POST returns 201 with a Location header.

## Task 2: Versioning, Pagination and Standardised Error Responses

Goal: Add API versioning, implement cursor/offset pagination, and standardise error formats.

Steps:

82.

Add versioning to your API URLs: change `/api/courses/` to `/api/v1/courses/`.

Discuss (in a code comment) two alternative versioning strategies: URL versioning vs header-based versioning (Accept: `application/vnd.api+json;version=1`).

83.

Implement offset pagination on GET `/api/v1/courses/`: accept `page` and `page_size` query params. Return a response envelope: `{'count': total, 'next': url_or_null, 'previous': url_or_null, 'results': [...]}`. This is the DRF pagination standard.

84.

Add a filtering query parameter `search=` to GET `/api/v1/courses/` that searches course name and code with a case-insensitive LIKE query.

85.

Standardise all error responses to follow the format: `{'error': {'code': 'NOT_FOUND', 'message': 'Course with id 99 does not exist', 'field': null}}`. Update all error handlers to use this format.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Python Backend Frameworks | Hands-On Exercise Book

Hint:

- 

URL versioning (`/v1/`) is simple and visible. Header versioning keeps URLs clean but is harder to test in a browser.

- 

Always include `count` and `next/previous` in paginated responses — clients need to know the total to build pagination controls.

Expected Outcome: GET `/api/v1/courses/?page=1&page_size=2` returns the correct envelope with `count`, `next`, `previous`, and `results`. Error responses follow the standardised format.

## OUTCOME:

The screenshot shows a REST client interface with the following components:

- GET /api/courses/** Get All Courses
- Parameters** section with a **Cancel** button.
- Name** and **Description** columns.
- limit** parameter set to **2** (integer, query).
- Execute** and **Clear** buttons.
- Responses** section.
- Curl** command: `curl -X 'GET' \ 'http://127.0.0.1:8000/api/courses/?limit=2' \ -H 'accept: application/json'`
- Request URL**: `http://127.0.0.1:8000/api/courses/?limit=2`
- Server response** section with **Code** and **Details** tabs.

The screenshot shows the server response details in the REST client interface:

- Server response** section with **Code** and **Details** tabs.
- Code**: 200
- Response body**:

```
[
  {
    "name": "Python Web Frameworks",
    "code": "CS301",
    "credits": 4,
    "id": 1,
    "department_id": 1
  },
  {
    "name": "Data Structures",
    "code": "CS231",
    "credits": 3,
    "id": 2,
    "department_id": 2
  }
]
```
- Response headers**:

```
content-length: 165
content-type: application/json
date: Thu, 02 Jul 2026 02:52:34 GMT
server: uvicorn
```
- Responses** section with **Code**, **Description**, and **Links** columns.
- Code**: 200
- Description**: Successful Response
- Links**: No links
- Media type**: application/json (dropdown menu)
- Controls**: Accept header.
- Example Value** | Schema